Worksheet EDA1 - WS05: Alocação Dinâmica II

Conteúdo: free, vazamentos, ponteiros pendentes e structs dinâmicas | Tempo previsto: 30-35 minutos | Modalidade: individual ou dupla

Objetivos de aprendizagem

- Explicar o efeito de free sobre um bloco alocado.
- Identificar ponteiros pendentes e vazamentos de memória.
- Representar estados antes e depois da liberação.
- Interpretar alocação dinâmica de structs e vetores de structs.
- Construir pequenos trechos que completam o ciclo alocar -> usar -> liberar.

Parte A - Compreensão

1. Marque V ou F e justifique as falsas:
( ) free(p) libera o bloco apontado por p.
( ) Depois de free(p), p passa automaticamente a valer NULL.
( ) Usar p[0] depois de free(p) é uma operação válida.
( ) Perder a última referência a um bloco sem chamar free pode produzir vazamento de memória.
( ) calloc dispensa a necessidade de free.

_____

2. Explique a diferença entre:
a) bloco liberado: _____
b) ponteiro com valor NULL: _____
c) ponteiro pendente: _____

Parte B - Simulação e representação

3. Considere os endereços simbólicos p@1000 e bloco@3000:

```
int *p;

p = calloc(3, sizeof(int));

if (p != NULL)
{
    p[1] = 8;
    free(p);
    p = NULL;
}
```

Represente a memória em três momentos:
a) imediatamente após calloc;

_____
_____

b) após p[1] = 8;

_____
_____

c) após free(p); p = NULL;

_____
_____

4. Analise:

```
int *p;

p = malloc(4 * sizeof(int));

if (p == NULL)
    return 1;

p = malloc(8 * sizeof(int));
```

O que acontece com o primeiro bloco? Por que o trecho caracteriza um vazamento?

_____
_____

Parte C - Leitura de código

5. Considere:

```
typedef struct
{
    int codigo;
    int quantidade;
} Registro;

Registro *p;

p = malloc(sizeof(Registro));

if (p != NULL)
{
    p->codigo = 10;
    p->quantidade = 4;
}
```

a) Por que o código usa p->codigo em vez de p.codigo?

_____

b) Desenhe p e o Registro alocado como regiões separadas.

_____
_____

6. Considere:

```
Registro *v;

v = calloc(3, sizeof(Registro));

if (v != NULL)
{
    v[0].codigo = 5;
    v[2].quantidade = 9;
}
```

Supondo sucesso, complete:

posição	codigo	quantidade
0		
1		
2		

Parte D - Construção

7. Complete o trecho para liberar corretamente o vetor e reduzir o risco de reutilização do endereço antigo:

```
free(               );

v =                ;
```

8. Complete a função abaixo sem escrever o restante da PC4:

```
Registro *criarRegistros(int n)
{
    Registro *v;

    v =                                          ;

    return v;
}
```

A função deve devolver um vetor dinâmico de n registros inicialmente zerados.

Parte E - Problema integrador

9. Considere:

```
Registro *v;
int n = 2;

v = calloc(n, sizeof(Registro));

if (v != NULL)
{
    v[0].codigo = 11;
    v[0].quantidade = 3;
    v[1].codigo = 22;
    v[1].quantidade = 7;

    free(v);
    v = NULL;
}
```

a) Represente o estado do vetor antes de free.

_____
_____

b) O que acontece com o bloco na chamada free(v)?

_____

c) Qual é o valor final de v?

_____

d) Seria válido calcular v[0].quantidade após o final do trecho? Justifique.

_____

Preparação para a prática

Na PC4 você utilizará um vetor dinâmico de Registro. Antes de sair, certifique-se de que consegue explicar:

1. quem guarda o endereço do bloco alocado;
2. quando o bloco deve ser liberado;
3. por que o ponteiro não deve ser usado após free;
4. como acessar os campos de uma posição v[i] de um vetor de structs.